Identity Protocols

Marc Müller

4tecture©
empower your software solutions



About me:

Marc Müller
Principal Consultant
@muellermarc

MVP
Microsoft Most Valuable Professional

4tecture©
empower your software solutions

Our Products:

Multi-Tenant OpenID Connect Identity Provider

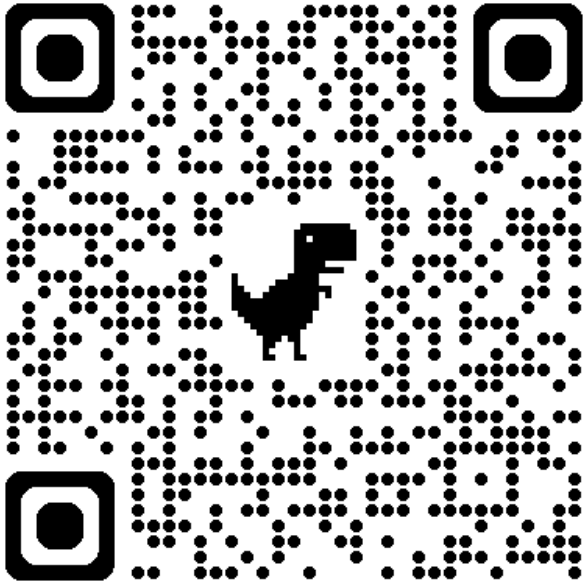
ProAuth
www.proauth.net

Enterprise Application Framework for .NET

ReaFx
www.reafx.net

Slide Download

https://www.4tecture.ch/events/dwx26-identityprotocols



Identity Protocols 2026

Beyond the OAuth Basics

- Sender-constrained tokens
- Browser token boundaries
- Machine-to-machine access
- High-assurance request flows

The One Rule

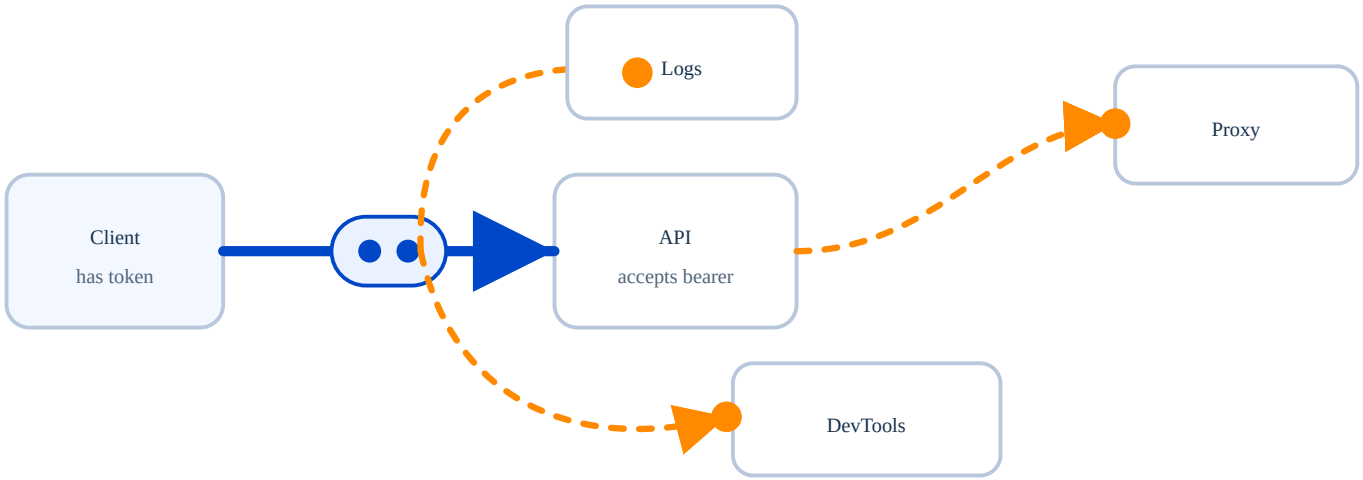
Stop designing for token possession.

Start designing for token context.

Why This Changed

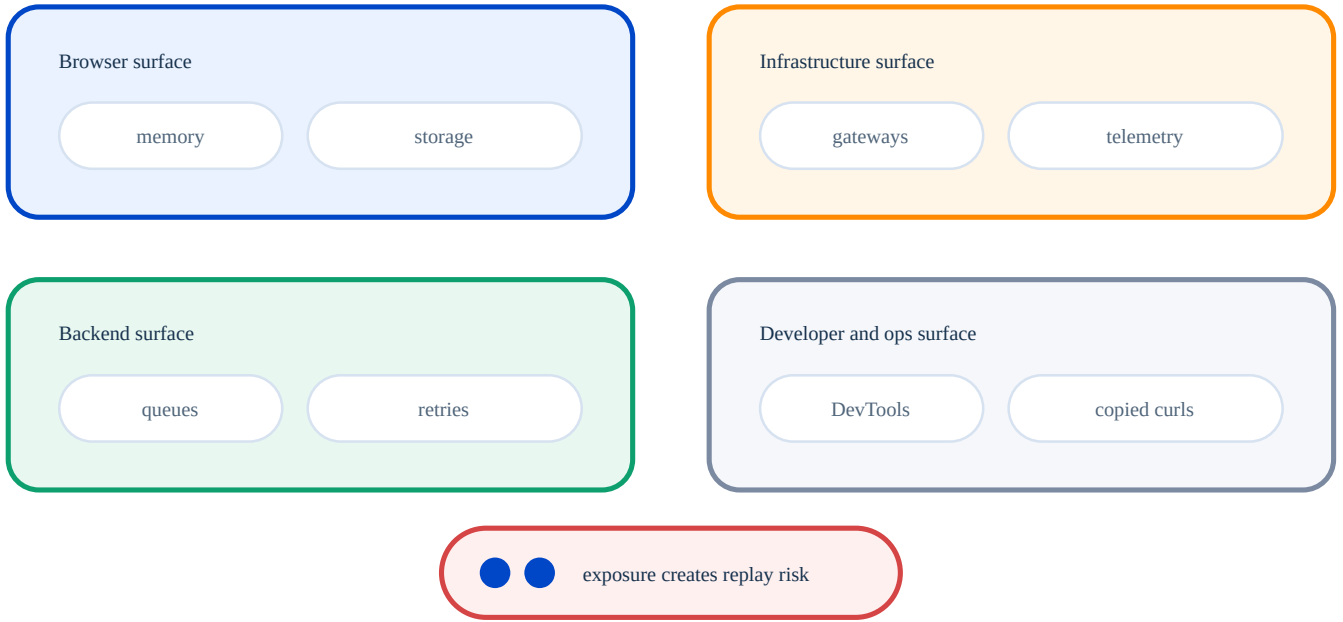
- Browser apps grew heavier
- APIs call more APIs
- Agents act without humans
- Bearer leakage is routine
- Zero-trust needs narrower claims

Token Leakage Is Boring



Bearer token replay starts with ordinary traces

Where Tokens Escape



Talk Map

Foundations

- 2026 OAuth baseline
- Why bearer replay matters
- Sender-constrained tokens

Architecture

- Browser token boundaries
- Agents and API chains
- High-assurance profiles
- Practical decision matrix

THE 2026 BASELINE

Baseline Checklist

- Authorization Code + PKCE
- Exact redirect URI matching
- No Implicit grant
- No password grant
- Issuer validation
- Refresh-token replay protection

What Got Demoted

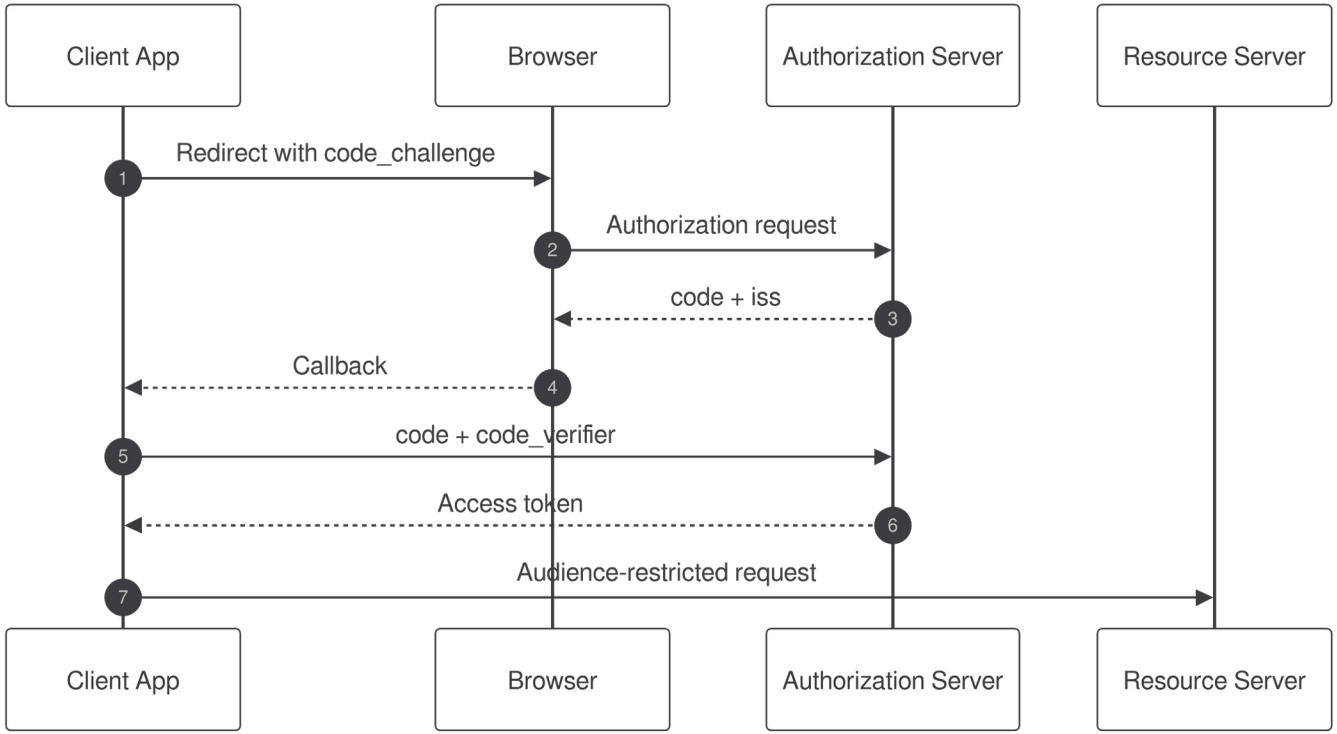
Old Habit

- Implicit in SPAs
- Password grant
- Loose redirects
- Shared client secrets

Modern Posture

- Code + PKCE
- Redirect-based login
- Exact redirect match
- Asymmetric proof

Baseline Flow



Mix-Up Defense

- Multi-issuer clients are exposed
- Store expected issuer per request
- Validate returned `iss`
- Prefer distinct redirect URIs too

Refresh Tokens

- Public clients need replay detection
- Rotate refresh tokens
- Or sender-constrain them
- Detect reuse as compromise

Demo



Security Baseline

Recap Demo 1: Security Baseline

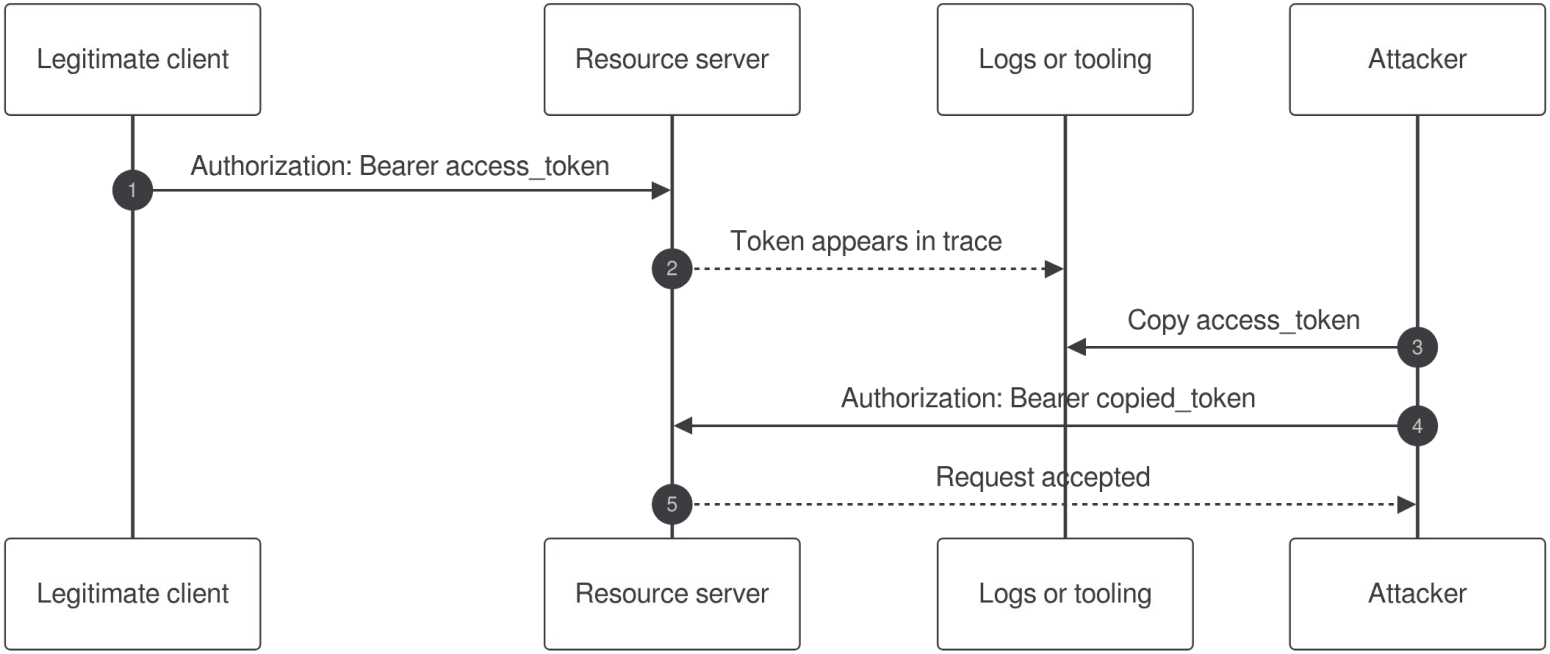
- Passkey sign-in
- Code + PKCE
- Callback includes `iss`
- Rotated refresh token
- API audience check

SENDER-CONSTRAINED TOKENS

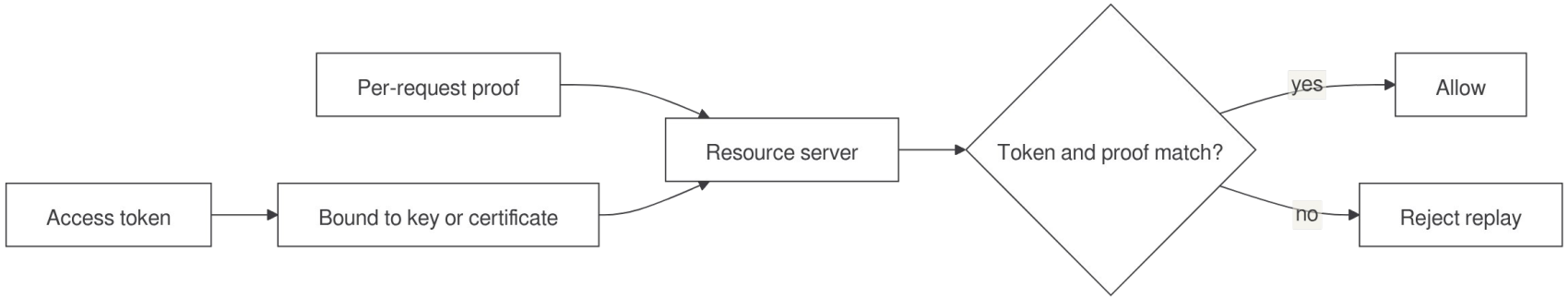
Bearer Tokens

- Possession is enough
- Logs become dangerous
- Proxies become dangerous
- Browser storage becomes dangerous
- Replay is the failure mode

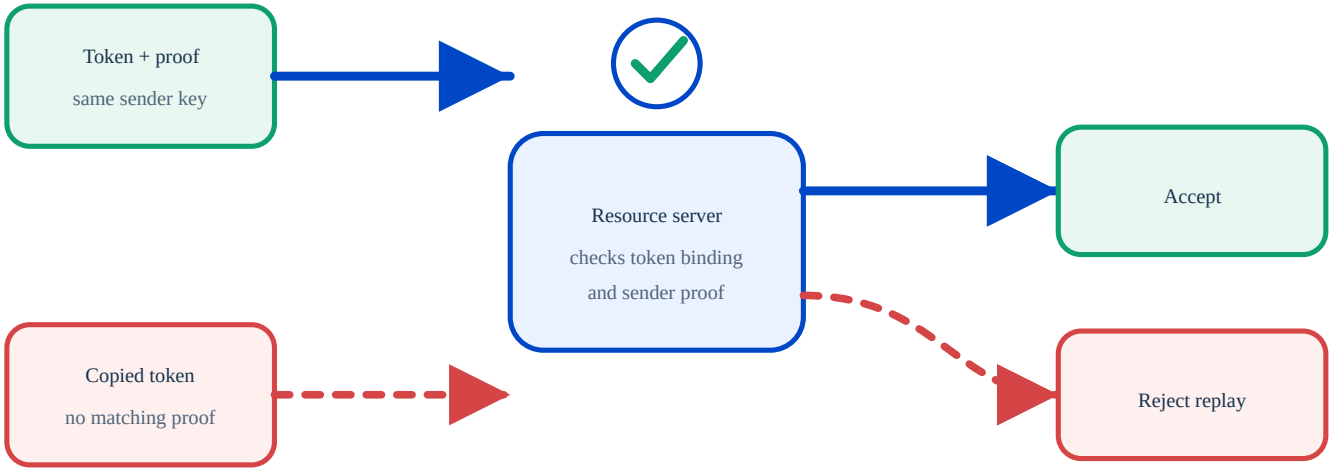
Bearer Replay Attack



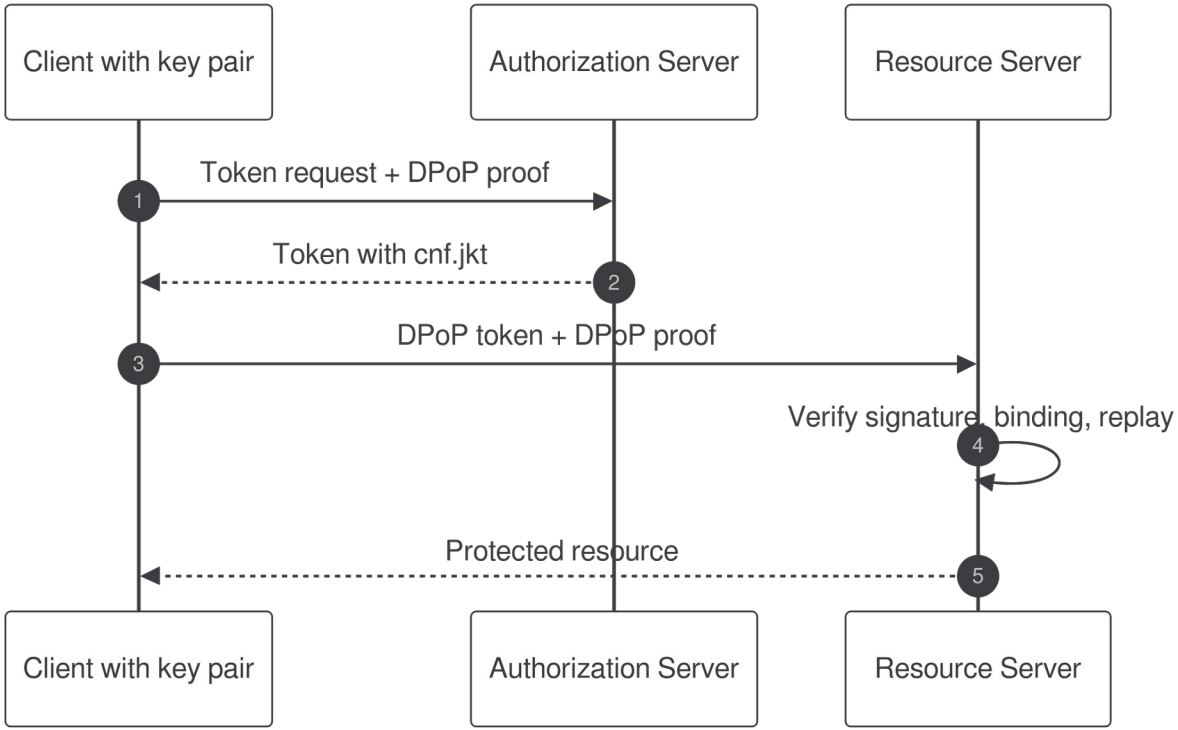
Proof Of Possession



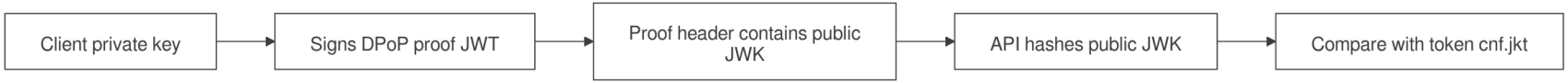
Sender Constraint Changes Replay



DPoP Flow


Client with key pair
Authorization Server
Resource Server
Token request + DPoP proof
1
Token with cnf.jkt
2
DPoP token + DPoP proof
3
Verify signature, binding, replay
4
Protected resource
5
Client with key pair
Authorization Server
Resource Server

Where Is The DPoP Proof?



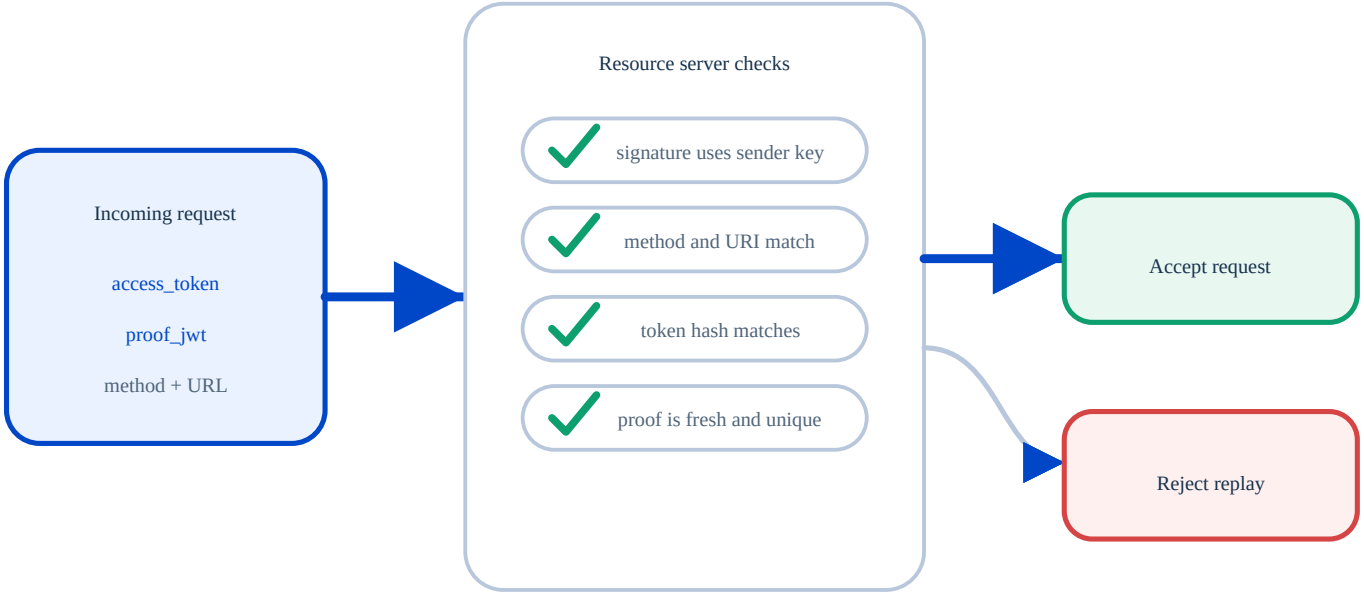
Where Is The DPoP Proof?

- Signature proves private-key possession
- `cnf.jkt` binds token to public key
- `ath` binds proof to this token
- `htm` and `htu` bind proof to request
- `jti` makes the proof one-time-use

DPoP Proof JWT Anatomy

```
DPoP proof JWT
|-- header
|   |-- typ: dpop+jwt
|   |-- alg: ES256
|   `-- jwk: public key
|-- payload
|   |-- htm: GET
|   |-- htu: https://payments-api.localtest.me/api/payments/dpop
|   |-- ath: hash(access token)
|   |-- jti: unique proof id
```

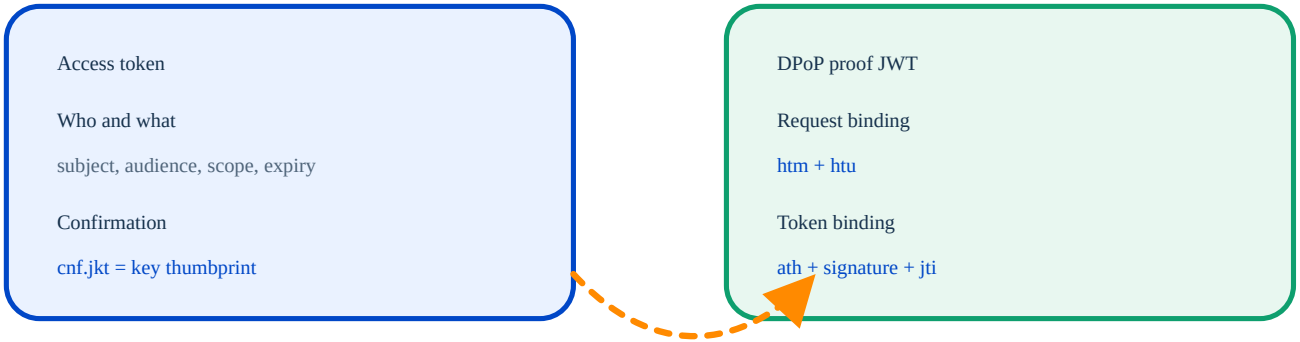
DPoP Checks


Incoming request
access_token
proof_jwt
method + URL
Resource server checks
signature uses sender key
method and URI match
token hash matches
proof is fresh and unique
Accept request
Reject replay

DPoP Data On The Wire

One protected API request

GET /orders/42 Authorization: DPoP access_token DPoP: proof_jwt

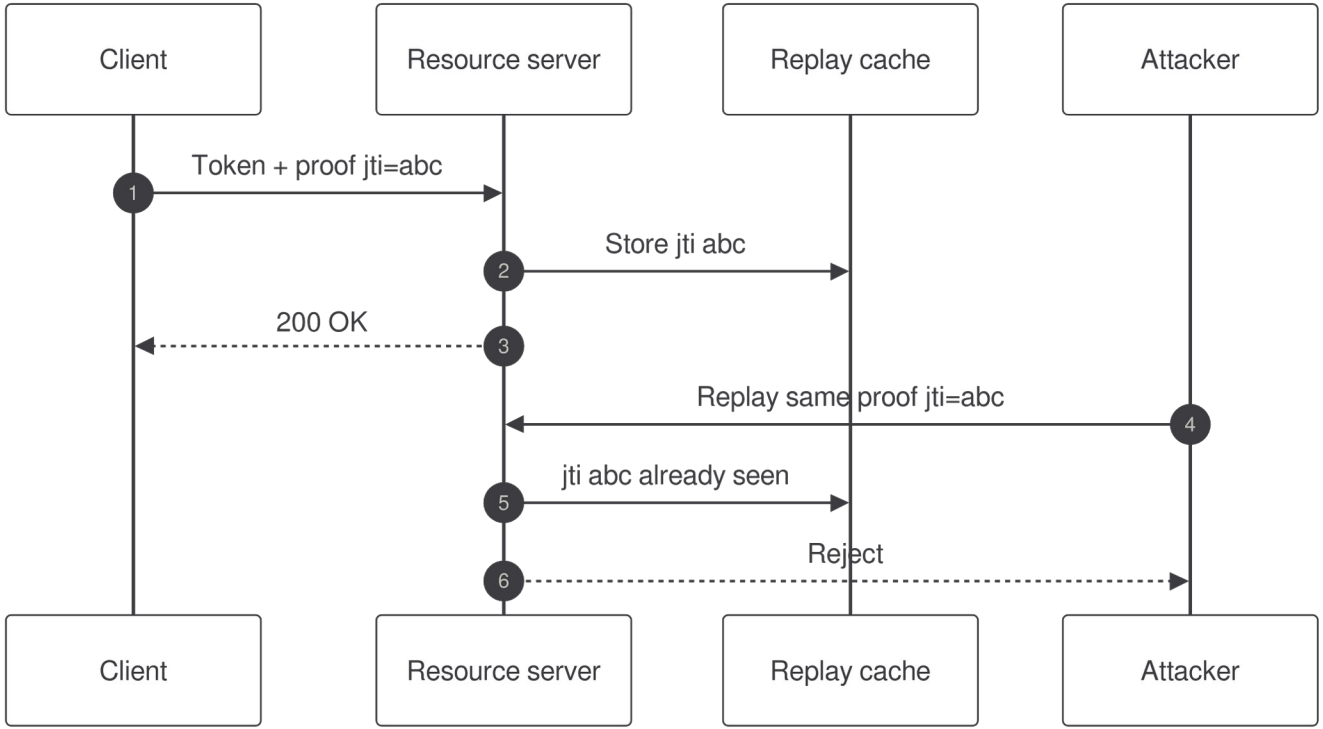


proof must match both the token and this HTTP request

DPoP Binding Data

```
Authorization: DPoP eyJhbGciOi...
   DPoP: eyJ0eXAiOiJkcG9wK2p3dCIs...
   {
     "token": { "cnf": { "jkt": "public-key-thumbprint" } },
     "proof": {
       "htm": "GET",
       "htu": "https://api.example/payments",
       "ath": "access-token-hash",
       "jti": "one-time-proof-id"
     }
   }
```

DPoP Replay Window



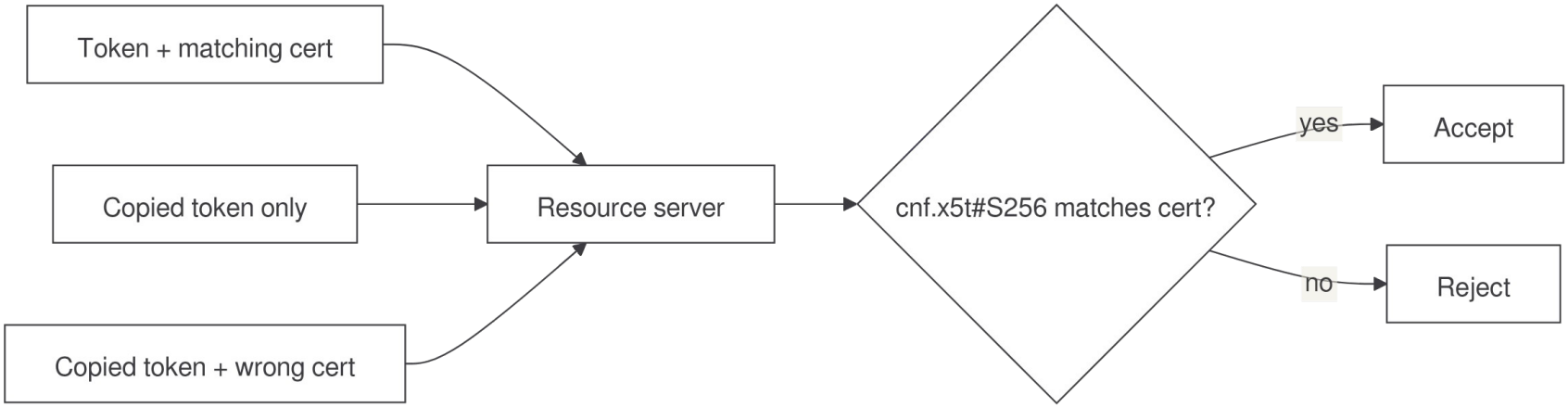
mTLS Token Binding

- Uses the TLS client certificate
- Binds token to certificate thumbprint
- Strong for server-to-server
- Operationally heavier than DPoP

mTLS Binding Data

```
{
  "token": {
    "cnf": {
      "x5t#S256": "certificate-sha256-thumbprint"
    }
  },
  "request": {
    "presentedClientCertificate": "same-certificate"
  }
}
```

mTLS Replay Test



DPoP Or mTLS?

Pattern	Best fit	Main trade-off
DPoP	Apps without TLS client certs	Key theft in runtime
mTLS	Controlled server environments	Certificate operations

Authorization Code Binding

- Bind code to DPoP key
- Use `dpop_jkt`
- PAR can carry DPoP proof
- Prevent code substitution paths


Demo
Replay Fails

Recap Demo 2: Replay Fails

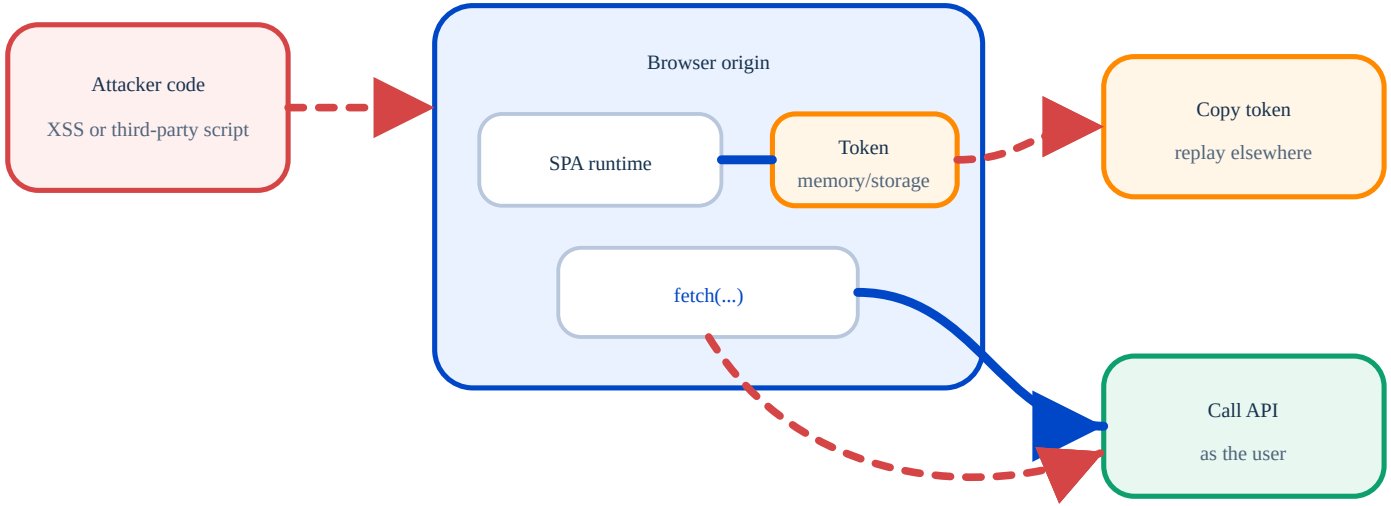
- Bearer replay succeeds
- DPoP replay fails
- mTLS replay fails
- Show `cnf` binding
- Show proof validation result

BROWSER APPS

Browser Threat Model

- JavaScript is the client
- Third-party scripts are common
- XSS changes everything
- PKCE protects the code
- PKCE does not hide tokens

Browser Token Exposure



PKCE protects the code exchange; it does not hide tokens after JavaScript can reach them.

Three Browser Patterns

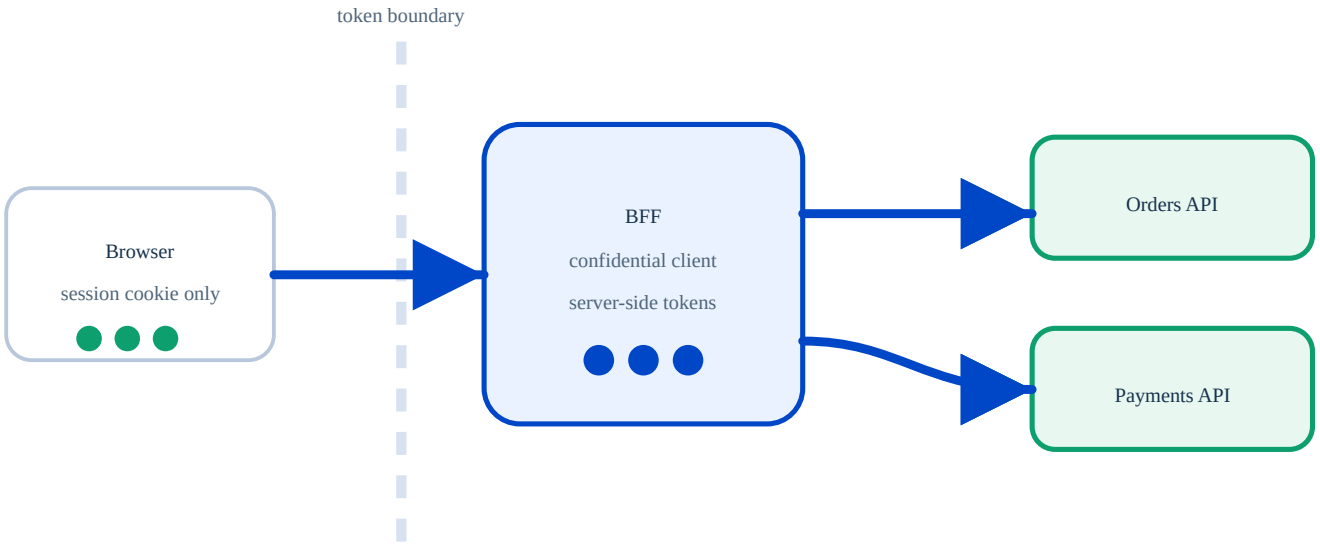
Stronger

- Full BFF
- Tokens stay server-side
- Browser uses session cookie

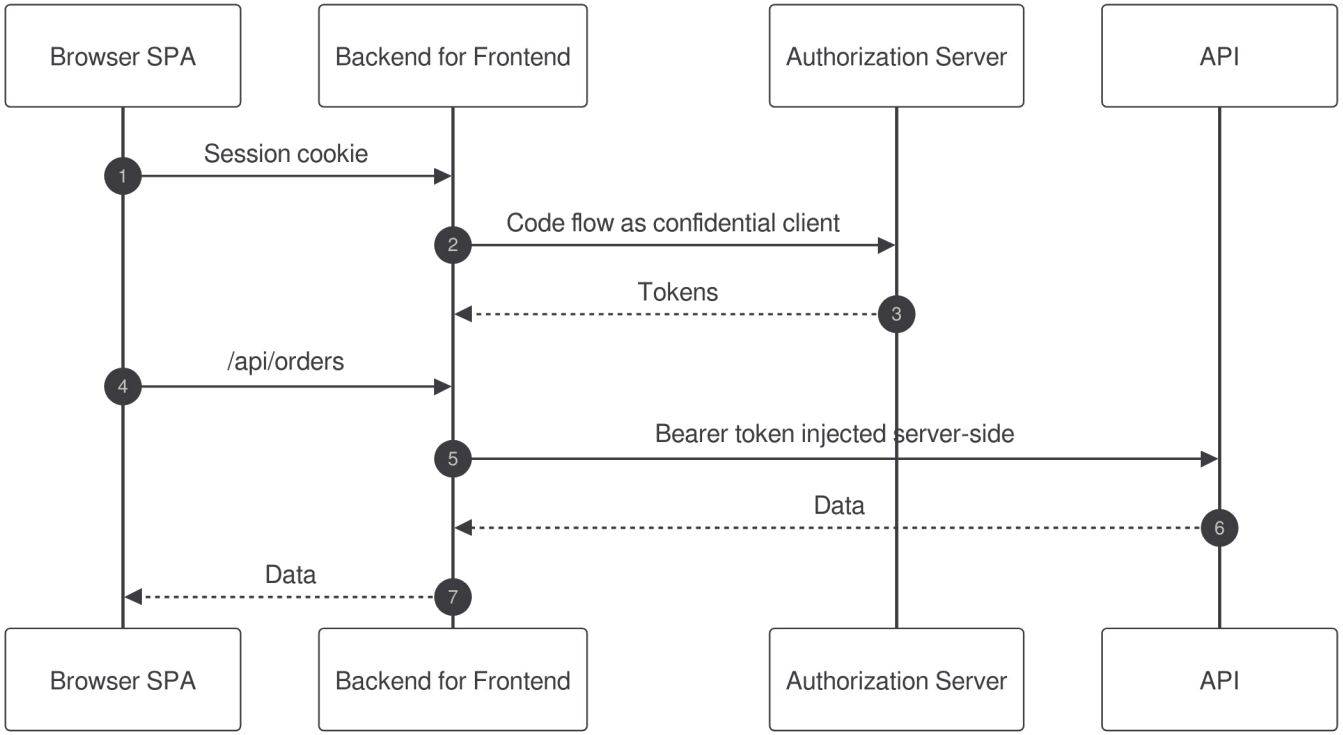
Weaker

- Token-mediating backend
- Browser gets access tokens
- Browser-only OAuth client

Move The Token Boundary



Full BFF


Browser SPA
Backend for Frontend
Authorization Server
API
Session cookie
1
Code flow as confidential client
2
Tokens
3
/api/orders
4
Bearer token injected server-side
5
Data
6
Data
7
Browser SPA
Backend for Frontend
Authorization Server
API

Same UI, Different Boundary

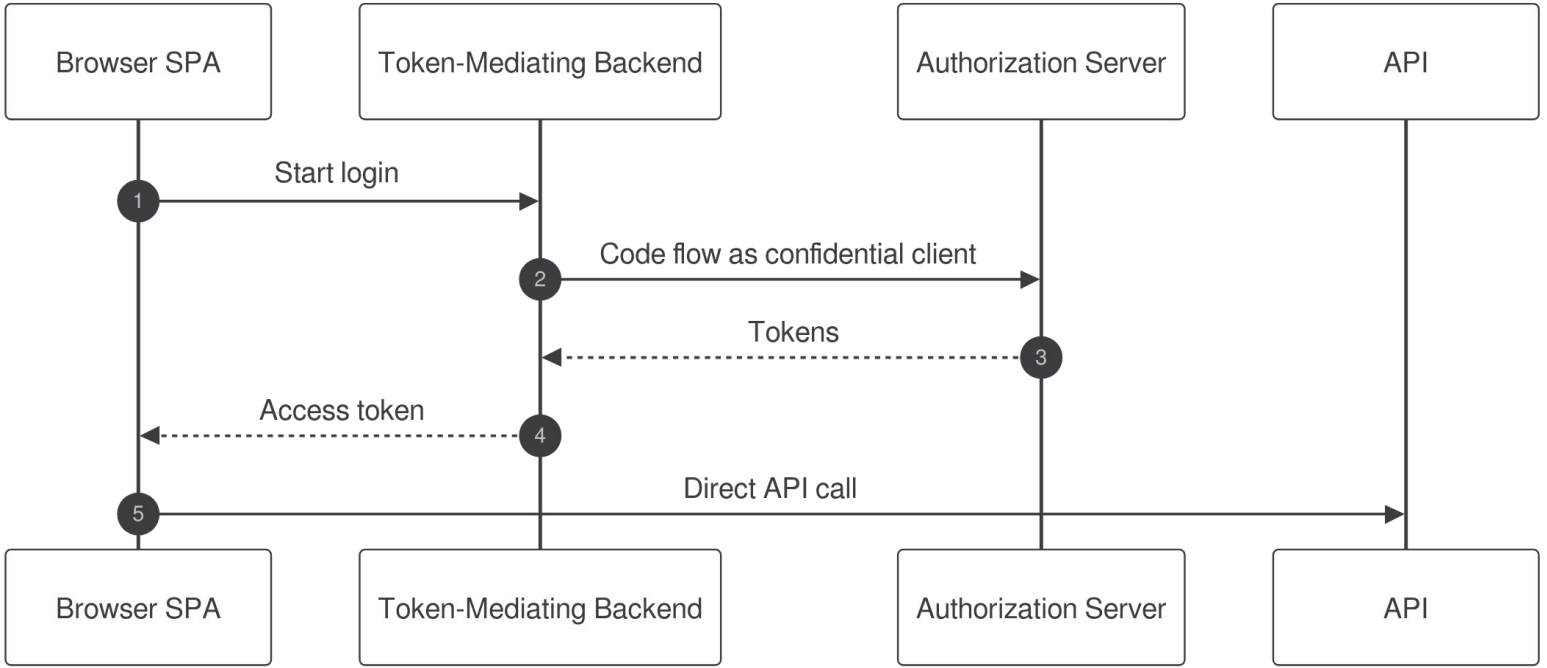
Direct SPA

- Browser receives token
- JavaScript can use token
- XSS can use token
- API sees browser caller

BFF

- Browser receives cookie
- Tokens stay server-side
- XSS lacks token
- API sees backend caller

Token-Mediating Backend



Browser Decision

- Can APIs be proxied?
- Is personal data involved?
- Are refresh tokens needed?
- Can same-origin cookies work?
- Is direct API latency critical?


Demo
Direct SPA vs BFF

Recap Demo 3: Direct SPA Vs BFF

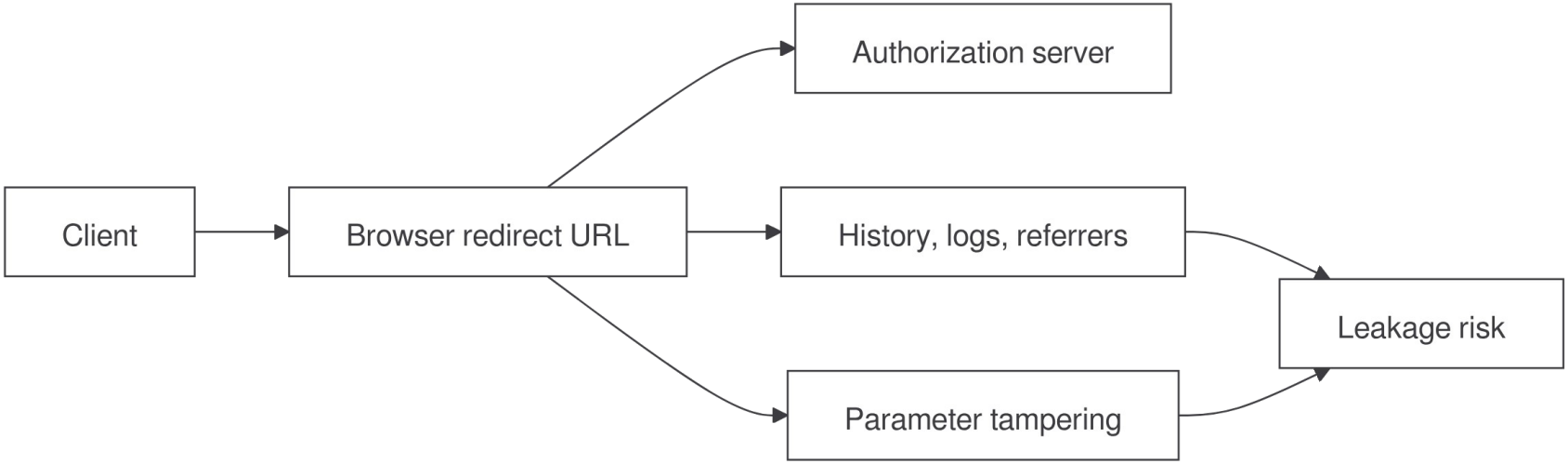
- Same UI, different boundary
- Direct SPA exposes token
- BFF exposes session cookie
- DevTools makes it visible
- API call still works

HIGH-ASSURANCE REQUEST FLOWS

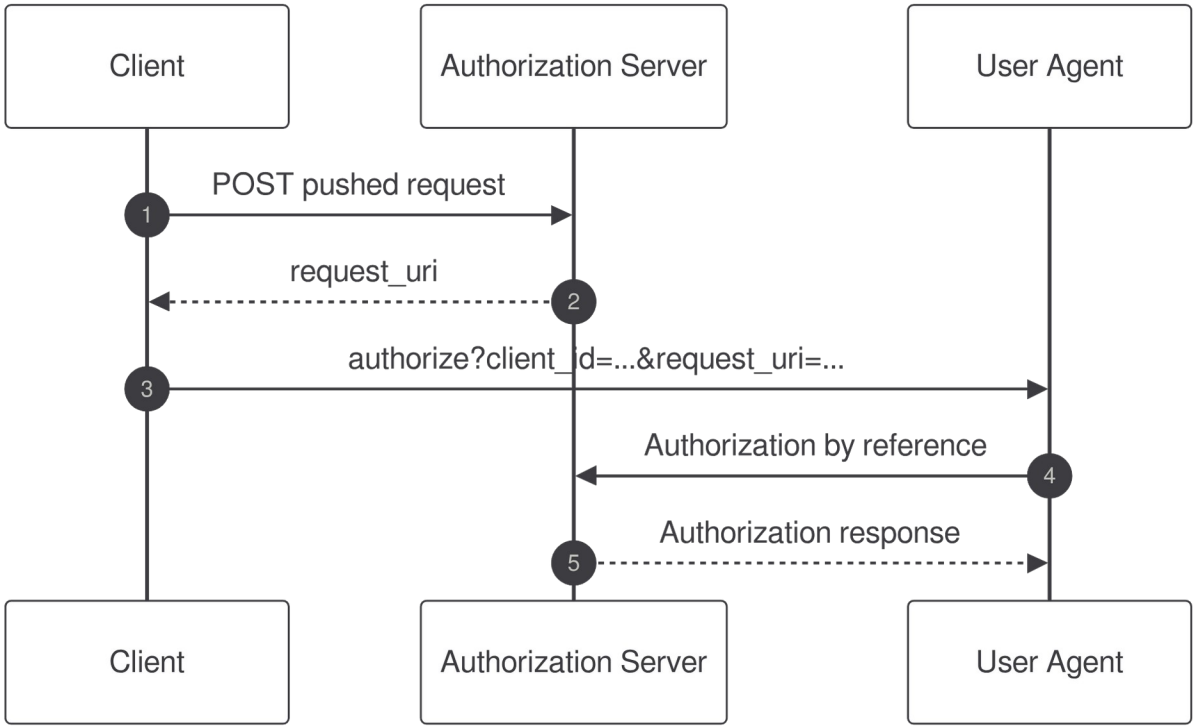
Front-Channel Pressure

- Long authorization URLs
- Sensitive request parameters
- Tampering opportunities
- Replayable request references
- Weak response assurance

Before PAR


Authorization server
Client
Browser redirect URL
History, logs, referrers
Leakage risk
Parameter tampering

PAR


Client
Authorization Server
User Agent
POST pushed request
1
request_uri
2
authorize?client_id=...&request_uri=...
3
Authorization by reference
4
Authorization response
5
Client
Authorization Server
User Agent

After PAR

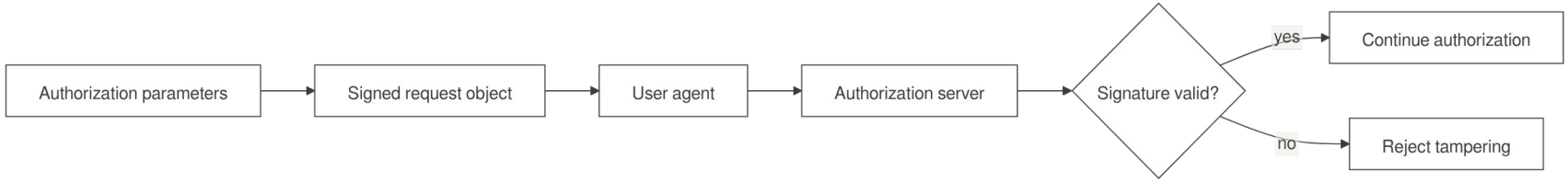
Back Channel

- Full request data
- Client authentication
- Early validation
- Stored by AS

Front Channel

- `client_id`
- `request_uri`
- Less URL exposure
- Less tamper surface

Signed Request Object



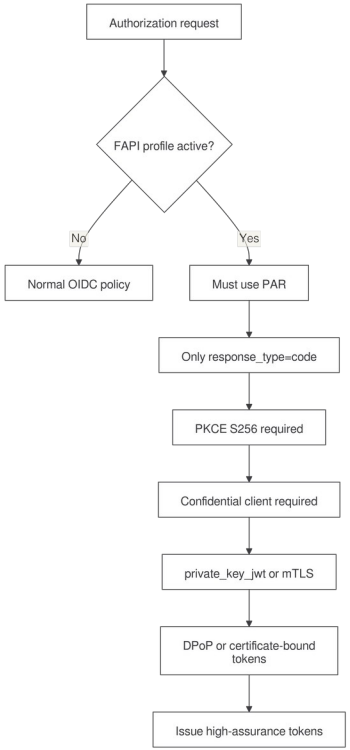
JAR, JARM, RAR

- JAR protects request objects
- JARM protects authorization responses
- RAR structures authorization details
- All reduce front-channel ambiguity

FAPI 2.0 Profile

- High-security OAuth profile
- Final specification, February 2025
- Builds on RFC 9700
- Requires sender-constrained tokens
- Uses PAR and PKCE

FAPI As Guardrail



FAPI Is Not A Checkbox

It enforces

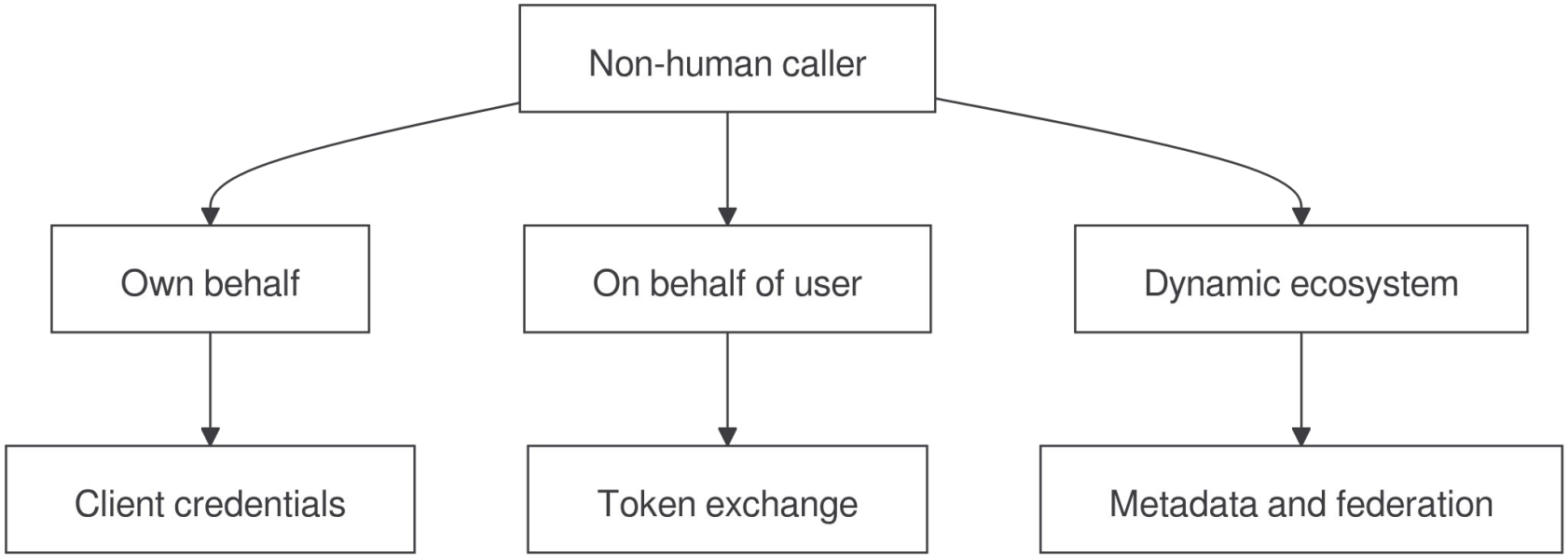
- PAR before authorization
- Code flow + PKCE S256
- Strong client authentication
- Sender-constrained tokens

It still needs

- Correct API authorization
- Key and certificate operations
- Resource validation
- Ecosystem governance

NON-HUMAN ACCESS

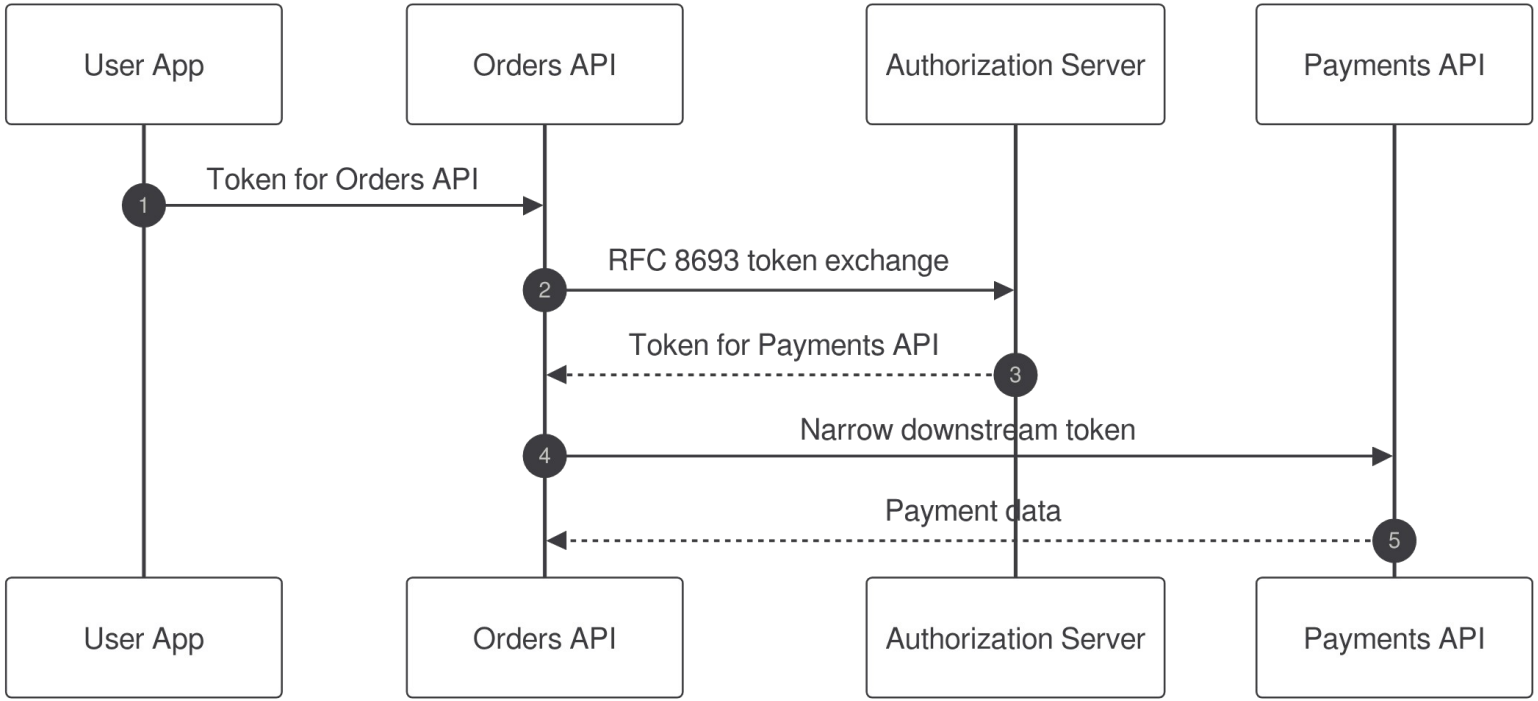
Name The Actor



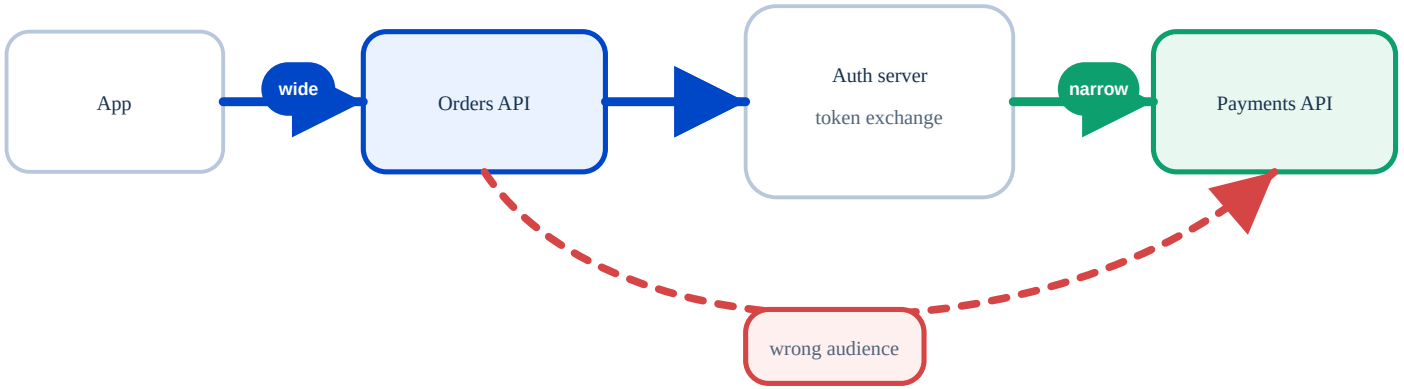
Strong Client Identity

- Prefer asymmetric authentication
- Use `private_key_jwt`
- Or use mTLS
- Avoid long-lived shared secrets
- Scope tokens narrowly

Token Exchange



Token Exchange Narrows The Hop



Exchange Request Data

Input

- `subject_token`
- `resource`
- `audience`
- Requested `scope`

Output

- New access token
- Downstream audience
- Narrower scope
- Preserved context

Resource Context

- Use `resource` indicators
- Validate aud
- Prefer narrow scopes
- Publish resource metadata
- Challenge with metadata URL

Validation Choices

JWT Tokens

- Local validation
- Standard claims
- Expire naturally

Opaque Tokens

- Introspection
- Central status
- Revocation friendly


Demo
Agent Chain

Recap Demo 4: Agent Chain

- Worker calls Orders API
- Orders needs Payments
- Original token rejected
- Exchange narrows audience
- Payments accepts exchanged token

ECOSYSTEMS AND POLICY

Specialized Tools

OpenID Federation

- Cross-organization trust
- Entity statements
- Ecosystem governance

AuthZEN API

- Externalized decisions
- Policy decision requests
- OAuth-adjacent authorization

DECISION MATRIX

Do First

Item	Priority	Effort
Code + PKCE	Critical	Low
Exact redirects	Critical	Low
Remove Implicit	Critical	Medium
Remove password grant	Critical	Medium
Validate `iss`	High	Low

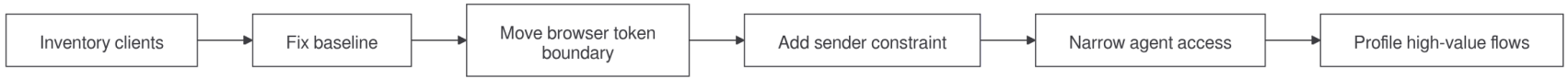
Upgrade Architecture

Item	Priority	Effort
BFF for serious SPAs	High	Medium
Refresh-token replay detection	High	Medium
Sender-constrained tokens	High	Medium
Strong client authentication	High	Medium
Token exchange	High	Medium

Adopt When Needed

Item	Best trigger
PAR	High-value authorization flows
JAR	Request integrity requirements
JARM	Response integrity requirements
RAR	Scopes are too coarse
FAPI 2.0	Regulated ecosystem profile
Federation	Cross-organization trust

Migration Backlog



Takeaways

- Bearer is the risk baseline
- PKCE is necessary, not sufficient
- Browser tokens need a boundary
- Agents need narrow downstream tokens
- High assurance starts before tokens

Thank you! Let’s continue the conversation

If you would like to discuss your challenges or explore how we can support you, please contact us.

4tecture GmbH
Industriestrasse 25
CH-8604 Volketswil
www.4tecture.ch

Marc Müller
Principal Consultant

4tecture©
empower your software solutions

ProAuth

ReaFx


4tecture©
empower your software solutions

APPENDIX

OAuth 2.0

- **Means:** Authorization framework
- **Benefit:** Delegated API access without sharing passwords
- **How:** Clients obtain access tokens for APIs after an authorization grant
- **Use when:** An app needs API access on behalf of a user, service, or workload

OIDC

- **Means:** OpenID Connect
- **Benefit:** Standardized user sign-in and identity claims
- **How:** Adds ID tokens, UserInfo, discovery, scopes, and claims to OAuth
- **Use when:** A client needs to authenticate a user, not only call an API

JWT, JWS, JWE

- **JWT:** JSON Web Token, a compact claims container
- **JWS:** Signed JWT, integrity and issuer authenticity
- **JWE:** Encrypted JWT, confidentiality of claims
- **Use when:** Tokens or protocol messages need portable cryptographic protection

PKCE

- **Means:** Proof Key for Code Exchange
- **Benefit:** Protects authorization codes from interception
- **How:** Authorization request sends a challenge; token request proves the verifier
- **Use when:** Authorization code flow is used, especially for public clients

RFC 9207 Issuer Identification

- **Means:** Authorization Server Issuer Identification
- **Benefit:** Helps prevent OAuth mix-up attacks
- **How:** Authorization responses include `iss`; clients compare it with expected issuer
- **Use when:** Clients interact with one or more authorization servers

OAuth Security BCP

- **Means:** OAuth 2.0 Security Best Current Practice
- **Benefit:** Modern baseline for safer OAuth deployments
- **How:** Recommends code + PKCE, exact redirects, issuer checks, replay protection, and sender constraint
- **Use when:** Designing or reviewing OAuth deployments in 2026

OAuth 2.1

- **Means:** Consolidation of OAuth 2.0 with current best practices
- **Benefit:** Removes old patterns from the mental default
- **How:** Centers authorization code + PKCE and drops implicit and password grants
- **Use when:** Explaining where modern OAuth is heading

Browser-Based Apps

- **Means:** OAuth guidance for browser applications
- **Benefit:** Names malicious JavaScript as a central threat
- **How:** Compares browser-only clients, token-mediating backends, and full BFFs
- **Use when:** Deciding where browser tokens should live

BFF

- **Means:** Backend for Frontend
- **Benefit:** Moves OAuth tokens out of browser JavaScript
- **How:** Browser holds an application session; backend stores tokens and calls APIs
- **Use when:** A browser app handles sensitive data and APIs can be proxied

DPoP

- **Means:** Demonstrating Proof of Possession
- **Benefit:** Stolen access token alone is not enough
- **How:** Token is bound to a public key thumbprint; each request carries a signed proof
- **Use when:** Sender constraint is needed but mTLS is impractical

mTLS

- **Means:** Mutual TLS client authentication and certificate-bound tokens
- **Benefit:** Strong sender constraint for controlled server environments
- **How:** Token contains `cnf.x5t#S256`; API compares it with the presented client certificate
- **Use when:** Backend services can operate client certificates safely

PAR

- **Means:** Pushed Authorization Requests
- **Benefit:** Keeps rich authorization request data out of the browser URL
- **How:** Client posts request details to the authorization server and receives `request_uri`
- **Use when:** Authorization requests are high-value, long, sensitive, or FAPI-style

JAR

- **Means:** JWT-Secured Authorization Request
- **Benefit:** Integrity and optional confidentiality for authorization request parameters
- **How:** Client sends request parameters inside a signed, optionally encrypted JWT
- **Use when:** Request parameters must not be changed silently in the front channel

JARM

- **Means:** JWT-Secured Authorization Response Mode
- **Benefit:** Integrity and optional confidentiality for authorization responses
- **How:** Authorization response parameters are returned inside a signed, optionally encrypted JWT
- **Use when:** The response itself needs cryptographic protection

RAR

- **Means:** Rich Authorization Requests
- **Benefit:** Expresses structured authorization details beyond flat scopes
- **How:** Clients send `authorization_details` with typed business objects
- **Use when:** Scopes are too coarse for payments, delegated actions, or fine-grained consent

FAPI 2.0

- **Means:** Financial-grade API Security Profile 2.0
- **Benefit:** Turns best practices into enforceable high-assurance policy
- **How:** Requires controls such as PAR, code flow, S256 PKCE, strong client authentication, and sender constraint
- **Use when:** API access is regulated, high-value, or ecosystem-governed

JWT Access Token Profile

- **Means:** Standard profile for JWT access tokens
- **Benefit:** Improves interoperability for self-contained access tokens
- **How:** Defines common claims and validation expectations for resource servers
- **Use when:** APIs validate JWT access tokens locally

Token Introspection

- **Means:** OAuth 2.0 Token Introspection
- **Benefit:** APIs can ask the authorization server about token state
- **How:** Resource server posts a token and receives metadata such as `active`, subject, scope, and audience
- **Use when:** Opaque tokens, central revocation, or live status checks are required

Token Revocation

- **Means:** OAuth 2.0 Token Revocation
- **Benefit:** Clients can explicitly invalidate tokens
- **How:** Client calls the revocation endpoint with a token and optional token type hint
- **Use when:** Logout, compromise, or client cleanup should invalidate refresh or access tokens

Token Exchange

- **Means:** OAuth 2.0 Token Exchange
- **Benefit:** Avoids forwarding broad or wrong-audience tokens downstream
- **How:** Client exchanges a subject token for another token with a different audience, resource, or scope
- **Use when:** APIs call APIs on behalf of users or workloads

Resource Indicators

- **Means:** OAuth 2.0 Resource Indicators
- **Benefit:** Helps clients request tokens for the right API
- **How:** Token requests include `resource` values that shape token audience
- **Use when:** One authorization server issues tokens for multiple APIs

Protected Resource Metadata

- **Means:** Metadata for OAuth protected resources
- **Benefit:** Helps clients discover how an API expects tokens
- **How:** Resource server publishes metadata such as authorization servers, scopes, and presentation methods
- **Use when:** APIs and clients participate in dynamic or multi-resource ecosystems

OpenID Federation

- **Means:** Federation framework for OpenID ecosystems
- **Benefit:** Scales trust establishment across organizations
- **How:** Entities publish signed statements that describe metadata, trust chains, and policies
- **Use when:** Trust is decentralized and cannot be managed as static client registrations

AuthZEN

- **Means:** OpenID Authorization API
- **Benefit:** Standardizes externalized authorization decision requests
- **How:** Applications ask a policy decision point whether a subject can perform an action on a resource
- **Use when:** Authorization decisions need central policy, auditability, or cross-service consistency

Passkeys And WebAuthn

- **Means:** Phishing-resistant public-key user authentication
- **Benefit:** Removes shared passwords from the login ceremony
- **How:** Authenticator signs a challenge scoped to the relying-party origin
- **Use when:** User authentication should resist credential phishing and password reuse

RFC References

- RFC 6749: https://www.rfc-editor.org/info/rfc6749/
- RFC 7515: https://www.rfc-editor.org/info/rfc7515/
- RFC 7516: https://www.rfc-editor.org/info/rfc7516/
- RFC 7519: https://www.rfc-editor.org/info/rfc7519/
- RFC 7636: https://www.rfc-editor.org/info/rfc7636/
- RFC 6750: https://www.rfc-editor.org/info/rfc6750/
- RFC 7009: https://www.rfc-editor.org/info/rfc7009/
- RFC 7662: https://www.rfc-editor.org/info/rfc7662/
- RFC 8693: https://www.rfc-editor.org/info/rfc8693/
- RFC 8705: https://www.rfc-editor.org/info/rfc8705/
- RFC 8707: https://www.rfc-editor.org/info/rfc8707/
- RFC 9068: https://www.rfc-editor.org/info/rfc9068/
- RFC 9101: https://www.rfc-editor.org/info/rfc9101/
- RFC 9126: https://www.rfc-editor.org/info/rfc9126/
- RFC 9207: https://www.rfc-editor.org/info/rfc9207/
- RFC 9396: https://www.rfc-editor.org/info/rfc9396/
- RFC 9449: https://www.rfc-editor.org/info/rfc9449/
- RFC 9700: https://www.rfc-editor.org/info/rfc9700/
- RFC 9728: https://www.rfc-editor.org/info/rfc9728/

References

- OAuth 2.1 draft: https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-15
- Browser-based apps draft: https://datatracker.ietf.org/doc/html/draft-ietf-oauth-browser-based-apps
- OpenID Connect Core: https://openid.net/specs/openid-connect-core-1_0.html
- FAPI 2.0 Security Profile: https://openid.net/specs/fapi-security-profile-2_0-final.html
- JARM: https://openid.net/specs/oauth-v2-jarm.html
- OpenID Federation 1.0: https://openid.net/specs/openid-federation-1_0.html
- OpenID Federation 1.1: https://openid.net/specs/openid-federation-1_1.html
- OpenID Authorization API 1.0: https://openid.net/specs/authorization-api-1_0.html
- W3C WebAuthn Level 3: https://www.w3.org/TR/webauthn-3/
- FIDO Passkeys: https://fidoalliance.org/passkeys/
- ProAuth BFF docs: https://docs.proauth.net/oidc-clients/01_Bff.html
- ProAuth OIDC client docs: https://docs.proauth.net/oidc-clients/02_OidcClient.html